Etherium_blockhain_exc_19

July 29, 2024

[898]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns
from datetime import datetime

from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error,␣
↪mean_absolute_percentage_error
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform
from scipy import stats
```

[899]:
```
TRAIN_DIRECTORY = "big-files/transactions100000.csv"
```

[900]:
```
# from pandas_gbq import read_gbq

# from google.cloud import bigquery
# from google.oauth2 import service_account


# KEY = "big-files/idyllic-tendril-421208-5bc1941f7008.json"

# credentials = service_account.Credentials.from_service_account_file(KEY)

# client = bigquery.Client(credentials=credentials, project=credentials.
↪project_id)

# table_id = 'bigquery-public-data.crypto_ethereum.transactions'
```

```
# query = f"""
# SELECT gas, max_fee_per_gas, max_priority_fee_per_gas,␣
↪receipt_effective_gas_price, gas_price, block_timestamp
# FROM `{table_id}`
# WHERE block_timestamp >= '2023-07-01'
# LIMIT 300000
# """

# data = read_gbq(query, project_id=credentials.project_id,␣
↪credentials=credentials)

# data.to_csv("xa.csv")





# #              BigQuery
# KEY = "big-files/idyllic-tendril-421208-5bc1941f7008.json"
# credentials = service_account.Credentials.from_service_account_file(KEY)
# client = bigquery.Client(credentials=credentials, project=credentials.
↪project_id)
# table_id = 'bigquery-public-data.crypto_ethereum.transactions'

# # SQL-
# query = f"""
# WITH data AS (
#     SELECT
#         EXTRACT(MONTH FROM block_timestamp) AS month,
#         AVG(gas) AS avg_gas,
#         AVG(max_fee_per_gas) AS avg_max_fee_per_gas,
#         AVG(max_priority_fee_per_gas) AS avg_max_priority_fee_per_gas,
#         AVG(receipt_effective_gas_price) AS avg_receipt_effective_gas_price,
#         AVG(gas_price) AS avg_gas_price
#     FROM `{table_id}`
#     WHERE block_timestamp BETWEEN '2023-01-01' AND '2023-07-31'
#     GROUP BY month
# )
# SELECT
#     months.month,
#     COALESCE(data.avg_gas, 0) AS avg_gas,
#     COALESCE(data.avg_max_fee_per_gas, 0) AS avg_max_fee_per_gas,
#     COALESCE(data.avg_max_priority_fee_per_gas, 0) AS␣
↪avg_max_priority_fee_per_gas,
#     COALESCE(data.avg_receipt_effective_gas_price, 0) AS␣
↪avg_receipt_effective_gas_price,
#     COALESCE(data.avg_gas_price, 0) AS avg_gas_price
```

```
# FROM
#     (SELECT 1 AS month UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT␣
 ↪4 UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7) AS months
# LEFT JOIN
#     data
# ON
#     months.month = data.month
# ORDER BY
#     months.month
# """

# #
# data = read_gbq(query, project_id=credentials.project_id,␣
 ↪credentials=credentials)

# #              CSV
# data.to_csv("xa.csv", index=False)
```

1 1. Data Exploration and Preprocessing

1.0.1 1.1 Loading the Dataset and Understanding its Structure

[901]:
```
# Load the data
data = pd.read_csv(TRAIN_DIRECTORY)
data
```

[901]:
```
                                                   hash  nonce  \
0       0xe762dd92f7a7bc16ee47188c4a2dfedc28202dad13c4…  57077
1       0x90cbf1652e6b93adff4b155bf843f59ecfb239c06340…     90
2       0x58ce377546086d72e6a9ee83b9a2c09d820159e35b9e…     28
3       0x2ebff83a86344dc821152c7be90ca36cca1cb1ecb042…    375
4       0x45c1a3de70daccd2f2d767d89a3b60887444fcd8abbf…      0
…                                                     …    …
99995   0x056ad81d0edbb64521b011d9e5d0d4188f6416f47886…     98
99996   0x6b321aecf9f7b7fdebba7850a6c499e6b14245d75974…    127
99997   0x9d326ec513a7ac96eb7ed54d32ba556b916be5a51cd2…      5
99998   0x38fc31f95e0322c1ff38be61a0f775b028d600d9a5d7…     11
99999   0xfd914eb222171b30eda4ce6fee899e769710866e5251…     69

        transaction_index                                from_address  \
0                     148  0xa83114a443da1cecefc50368531cace9f37fcccb
1                     113  0xeb549edf318fcbbed23cb3dd2323449e401ebd3c
2                      16  0x1225c2590a335b5dc20a67a07cf045245f463a4a
3                      49  0xa0a93347c540860d2ffc4891906cd1b8c2ccb921
4                       9  0x0c9fdef04b350820c035174e7f755de7af169fc0
…                     …                                          …
99995                  94  0xcee4011e2e90d65a94b2ac5ed596c9834a607767
```

```
99996                  6  0x3a0e3aa2524adff222a57aa611fd62a31065ad34
99997                140  0x1db430f53d164e5a1fb431d66b7c2589b5a155c3
99998                124  0x806633d9d0c353802a5309885ded1aa6f6e0e2e6
99999                 39  0x6e4ca415040a270a1e03c2ea6cc0c2b531de5d12

                                     to_address         value     gas  \
0      0x4675c7e5baafbffbca748158becba61ef3b0a263  2.027349e+17   21000
1      0xde8f22fbc53add5da02cf75616ac8f1eeda07c68  4.390000e+18   21000
2      0xa5f565650890fba1824ee0f21ebbbf660a179934  7.798510e+14   49369
3      0xd37bbe5744d730a1d98d8dc97c42f0ca46ad7146  0.000000e+00  450000
4      0x077d360f11d220e4d5d831430c81c26c9be7c4a4  1.563463e+16   22000
…                                            …           …     …
99995  0x3fc91a3afd70395cd496c647d5a6cc9d4b2b7fad  0.000000e+00  241011
99996  0x7a250d5630b4cf539739df2c5dacb4c659f2488d  0.000000e+00  315324
99997  0xdac17f958d2ee523a2206206994597c13d831ec7  0.000000e+00   63507
99998  0x3fc91a3afd70395cd496c647d5a6cc9d4b2b7fad  0.000000e+00  274021
99999  0xf3de3c0d654fda23dad170f0f320a92172509127  0.000000e+00  244850

        gas_price                                                 input  \
0      2357096623                                                    0x
1      2648096623                                                    0x
2      3847018376                                             0x02777bd5
3      4139045669  0x1b738b32000000000000000000000000d37bbe5744d7…
4      3000000000                                                    0x
…           …                                                       …
99995  2842724744  0x3593564c000000000000000000000000000000000000…
99996  8967031119  0x791ac947000000000000000000000000000000000000…
99997  1828031119  0xa9059cbb000000000000000000000000500b81835128…
99998  1828031119  0x3593564c000000000000000000000000000000000000…
99999  2809738399  0x9871efa400000000000000000000000008802269d1283…

       receipt_cumulative_gas_used  …  block_number  \
0                         15889746  …      20390203
1                         13441387  …      20390203
2                           921021  …      20390204
3                          5412686  …      20390205
4                          1384059  …      20390205
…                            …  …           …
99995                      8325646  …      20302100
99996                      1128992  …      20302101
99997                     16451783  …      20302101
99998                     14563342  …      20302101
99999                      7101117  …      20302101

                                          block_hash  max_fee_per_gas  \
0      0x50251abb442a34c796c7d04515c5c5168eca258b0f64…     2.357097e+09
1      0x50251abb442a34c796c7d04515c5c5168eca258b0f64…     3.817832e+09
```

```
2      0x78fb6920b2a3d2578d9ba84be279f01a136f27e513f5…     3.873482e+09
3      0x3dce7866de8d6bcbe2cf2decf41eeb9fa73880394f93…     1.000000e+10
4      0x3dce7866de8d6bcbe2cf2decf41eeb9fa73880394f93…              NaN
…                                                   …               …
99995  0x81d87f650570e14f6ccb94446142e5de15b010bc400e…     3.453106e+09
99996  0xa2b5cd7d9ffab6f1911449d3798f01063612c6e62d19…     1.457000e+10
99997  0xa2b5cd7d9ffab6f1911449d3798f01063612c6e62d19…     2.157093e+09
99998  0xa2b5cd7d9ffab6f1911449d3798f01063612c6e62d19…     2.009208e+09
99999  0xa2b5cd7d9ffab6f1911449d3798f01063612c6e62d19…     2.990287e+09

       max_priority_fee_per_gas transaction_type  receipt_effective_gas_price  \
0                  0.000000e+00                2                    2357096623
1                  2.910000e+08                2                    2648096623
2                  1.472445e+09                2                    3847018376
3                  2.000000e+09                2                    4139045669
4                           NaN                0                    3000000000
…                             …              …                             …
99995              9.700000e+08                2                    2842724744
99996              7.140000e+09                2                    8967031119
99997              1.000000e+06                2                    1828031119
99998              1.000000e+06                2                    1828031119
99999              9.827073e+08                2                    2809738399

      max_fee_per_blob_gas  blob_versioned_hashes  receipt_blob_gas_price  \
0                      NaN                     []                     NaN
1                      NaN                     []                     NaN
2                      NaN                     []                     NaN
3                      NaN                     []                     NaN
4                      NaN                     []                     NaN
…                        …                      …                       …
99995                  NaN                     []                     NaN
99996                  NaN                     []                     NaN
99997                  NaN                     []                     NaN
99998                  NaN                     []                     NaN
99999                  NaN                     []                     NaN

       receipt_blob_gas_used
0                        NaN
1                        NaN
2                        NaN
3                        NaN
4                        NaN
…                          …
99995                    NaN
99996                    NaN
99997                    NaN
99998                    NaN
```

```
99999                     NaN

[100000 rows x 25 columns]
```

[902]:
```
# | **Column Name**                    | **Description**                                                  ␣
↪                                                                                                          |
#␣
↪|-----------------------------------|-----------------------------------------------------------------------
# | 7. `gas`                           | The maximum amount of gas that the␣
↪sender is willing to pay for the transaction to be executed.                |
# | 8. `gas_price`                     | The price per unit of gas in Wei that␣
↪the sender is willing to pay.                                                |
# | 15. `block_timestamp`              | The timestamp of the block, indicating␣
↪when the block was mined.                                                    |
# | 18. `max_fee_per_gas`              | The maximum fee per unit of gas that␣
↪the sender is willing to pay (includes both base fee and priority fee).  |
# | 19. `max_priority_fee_per_gas`     | The maximum priority fee per unit of␣
↪gas that the sender is willing to pay to expedite the transaction.       |
# | 21. `receipt_effective_gas_price`  | The effective gas price paid for the␣
↪transaction, taking into account the current base fee and priority fee.  |
```

[903]:
```
# We analyze and understand that we only need 6 columns
df = data[["gas", "max_fee_per_gas", "max_priority_fee_per_gas",␣
↪"receipt_effective_gas_price", "block_timestamp", "gas_price",]]
```

[904]:
```
# View the first few rows
df.head()
```

[904]:
```
      gas  max_fee_per_gas  max_priority_fee_per_gas  \
0   21000     2.357097e+09              0.000000e+00
1   21000     3.817832e+09              2.910000e+08
2   49369     3.873482e+09              1.472445e+09
3  450000     1.000000e+10              2.000000e+09
4   22000              NaN                       NaN

   receipt_effective_gas_price            block_timestamp   gas_price
0                   2357096623  2024-07-26 10:43:35+00:00  2357096623
1                   2648096623  2024-07-26 10:43:35+00:00  2648096623
2                   3847018376  2024-07-26 10:43:47+00:00  3847018376
3                   4139045669  2024-07-26 10:43:59+00:00  4139045669
4                   3000000000  2024-07-26 10:43:59+00:00  3000000000
```

[905]:
```
# We see information about the dimension of our dataframe
df.shape
```

[905]:
```
(100000, 6)
```

[906]:
```python
# Information about the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 6 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   gas                          100000 non-null  int64
 1   max_fee_per_gas              95858 non-null   float64
 2   max_priority_fee_per_gas     95858 non-null   float64
 3   receipt_effective_gas_price  100000 non-null  int64
 4   block_timestamp              100000 non-null  object
 5   gas_price                    100000 non-null  int64
dtypes: float64(2), int64(3), object(1)
memory usage: 4.6+ MB
```

[907]:
```python
# Statistics for numerical data
df.describe()
```

[907]:
```
                gas  max_fee_per_gas  max_priority_fee_per_gas  \
count  1.000000e+05     9.585800e+04              9.585800e+04
mean   2.370734e+05     2.164203e+10              3.070349e+09
std    7.228919e+05     9.435149e+10              6.978934e+10
min    2.100000e+04     1.258426e+09              0.000000e+00
25%    3.180600e+04     4.453688e+09              4.703962e+07
50%    9.456100e+04     1.148593e+10              5.000000e+08
75%    2.445075e+05     1.879395e+10              2.000000e+09
max    2.850000e+07     2.001124e+13              2.000000e+13

       receipt_effective_gas_price     gas_price
count                 1.000000e+05  1.000000e+05
mean                  1.025560e+10  1.025560e+10
std                   6.849728e+10  6.849728e+10
min                   1.258426e+09  1.258426e+09
25%                   3.320239e+09  3.320239e+09
50%                   6.339680e+09  6.339680e+09
75%                   1.271655e+10  1.271655e+10
max                   2.001124e+13  2.001124e+13
```

[908]:
```python
# the number of unique values for each column
df.nunique()
```

[908]:
```
gas                           36617
max_fee_per_gas               51233
max_priority_fee_per_gas      14174
receipt_effective_gas_price   54029
block_timestamp                6328
```

```
gas_price                       54029
dtype: int64
```

1.0.2 1.2. Perform basic data cleaning, including handling missing values and removing duplicates

[909]:
```
# Check for missing values
df.isnull().sum()
```

[909]:
```
gas                              0
max_fee_per_gas               4142
max_priority_fee_per_gas      4142
receipt_effective_gas_price      0
block_timestamp                  0
gas_price                        0
dtype: int64
```

[910]:
```
# Check for duplicates and non-dublicates
df.duplicated().value_counts()
```

[910]:
```
False    92297
True      7703
Name: count, dtype: int64
```

[911]:
```
# Remove duplicates
df = df.drop_duplicates()
```

[912]:
```
# replacing all missing values with mean values
numeric_columns = df.select_dtypes(include=[np.number]).columns # select only␣
 ↪numeric columns
df[numeric_columns] = df[numeric_columns].fillna(df[numeric_columns].mean())
```

[913]:
```
# Rename block_timestamp in date
df = df.rename(columns={"block_timestamp": "date"})
# Rename gas in gas_Value
df = df.rename(columns={"gas": "gas_value"})
```

[914]:
```
# Converting a date to a normal format
df["date"] = pd.to_datetime(df["date"])
```

[915]:
```
# Lets make sure that the conversion was successful
df["date"].head()
```

[915]:
```
0   2024-07-26 10:43:35+00:00
1   2024-07-26 10:43:35+00:00
2   2024-07-26 10:43:47+00:00
3   2024-07-26 10:43:59+00:00
```

```
4   2024-07-26 10:43:59+00:00
5   2024-07-26 10:43:59+00:00
6   2024-07-26 10:43:59+00:00
Name: date, dtype: datetime64[ns, UTC]
```

[916]:
```python
# Transformation wei in gwei. So that everything is in the same currency
def wei_to_gwei(wei):
    return wei / 1e9
```

[917]:
```python
# We apply the conversion to the columns so that everything is in the same␣
↪price format and one type
df.loc[:, "max_fee_per_gas"] = df["max_fee_per_gas"].apply(wei_to_gwei).
↪astype(float)
df.loc[:, "max_priority_fee_per_gas"] = df["max_priority_fee_per_gas"].
↪apply(wei_to_gwei).astype(float)
df.loc[:, "receipt_effective_gas_price"] = df["receipt_effective_gas_price"].
↪apply(wei_to_gwei).astype(float)
```

```
C:\Users\ivanz\AppData\Local\Temp\ipykernel_5480\2792408785.py:4: FutureWarning:
Setting an item of incompatible dtype is deprecated and will raise in a future
error of pandas. Value '[2.35709662 2.64809662 3.84701838 … 1.82803112
1.82803112 2.8097384 ]' has dtype incompatible with int64, please explicitly
cast to a compatible dtype first.
  df.loc[:, "receipt_effective_gas_price"] =
df["receipt_effective_gas_price"].apply(wei_to_gwei).astype(float)
```

[918]:
```python
# detecting anomalies in gas_price
z_scores = stats.zscore(df["gas_price"])
anomalies = df[abs(z_scores) > 3] # if the indicator is < -3 or > 3, then this␣
↪is anamaly
anomalies
```

[918]:
```
       gas_value  max_fee_per_gas  max_priority_fee_per_gas  \
2948       87086       473.250126                473.250126
5885       21000       318.854609                318.854609
5996      697800       357.262095                357.262095
7088      277732       268.195259                265.688010
10870     117532      2146.304707               2144.089171
…          …              …                         …
94383     165692       227.515000                227.515000
94762     474424       310.257781                310.257781
94982    1000000       282.383696                275.906339
95849     156090       342.184403                342.184403
96880     347060      1111.000000               1111.000000

       receipt_effective_gas_price                      date      gas_price
2948                    473.250126 2024-07-26 10:16:35+00:00   473250126239
```

```
5885                    318.854609 2024-07-26 10:29:11+00:00   318854609036
5996                    357.262095 2024-07-26 10:43:35+00:00   357262094833
7088                    268.195259 2024-07-26 10:30:23+00:00   268195258757
10870                  2146.304707 2024-07-26 10:40:59+00:00  2146304706931
…                            …                       …            …
94383                   227.515000 2024-07-14 01:34:47+00:00   227514999799
94762                   310.257781 2024-07-14 01:42:47+00:00   310257781246
94982                   279.145018 2024-07-14 01:47:11+00:00   279145017507
95849                   342.184403 2024-07-14 02:05:11+00:00   342184402526
96880                  1111.000000 2024-07-14 02:28:35+00:00  1111000000000

[120 rows x 6 columns]
```

[919]:
```
# deleting abnormal rows (their number is small)
df = df[~(abs(z_scores) > 3)]
```

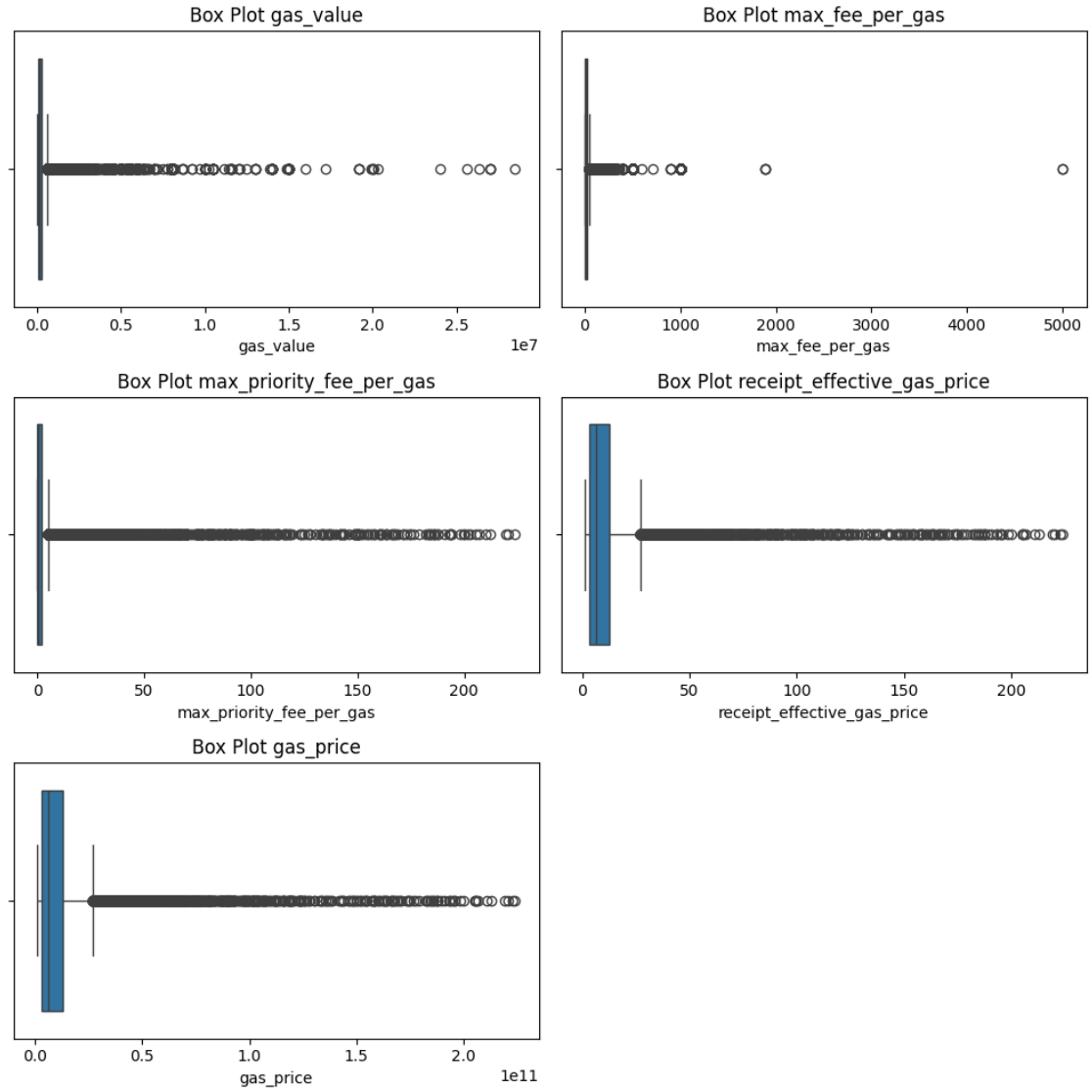
[920]:
```
# Create label. label = y
label = df.loc[:, "gas_price"].apply(wei_to_gwei).astype(float)
```

[921]:
```
# Making sure that the indexes in df and label match
df, label = df.align(label, join="inner", axis=0)
```

1.0.3 1.3. Generate summary statistics and visualize the distributions of key features

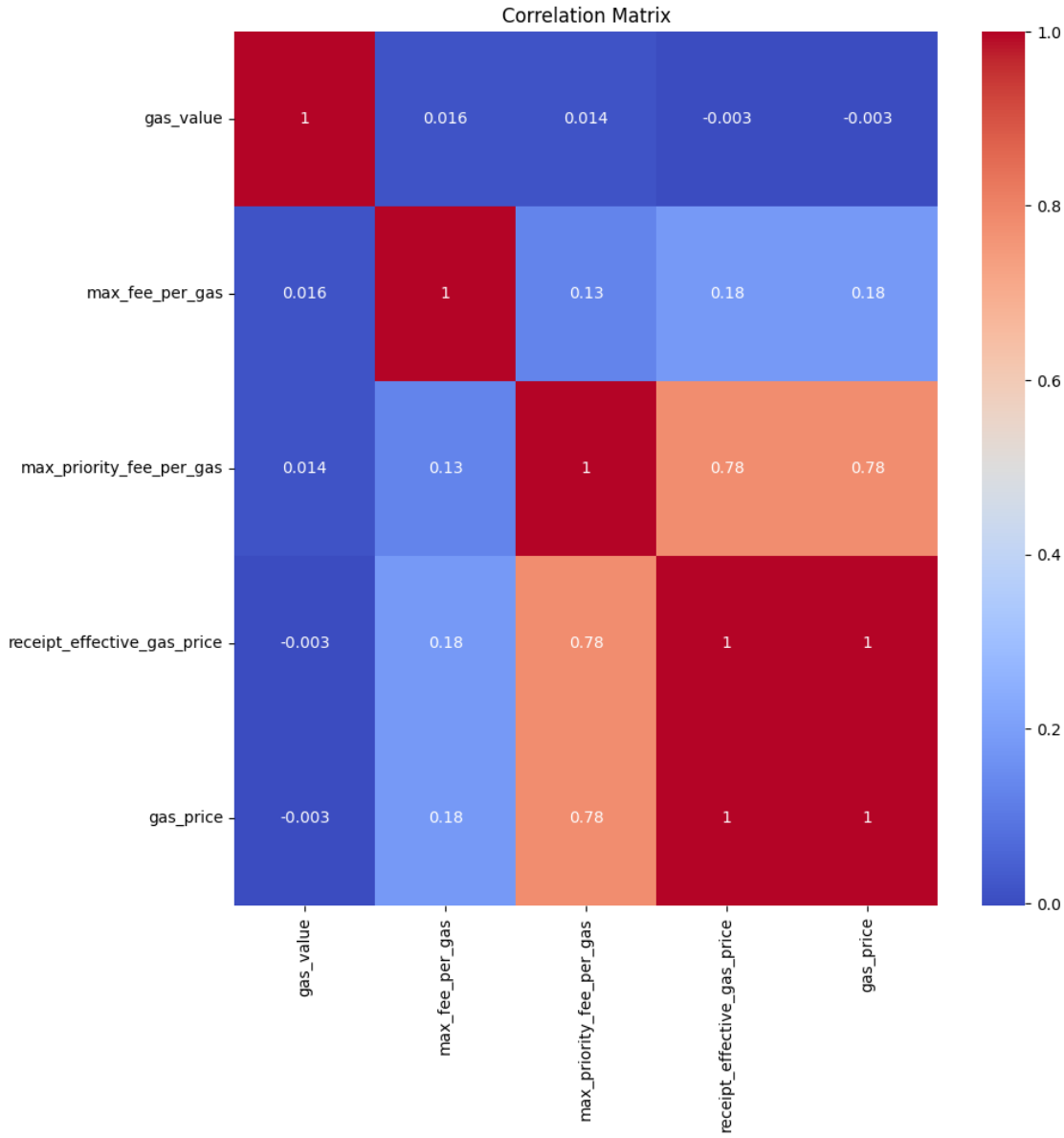
[922]:
```
# Key features
key_features = ["gas_value", "max_fee_per_gas", "max_priority_fee_per_gas",␣
 ↪"receipt_effective_gas_price", "gas_price"]
```

[923]:
```
# Boxplot for key features
plt.figure(figsize=(10, 10))

for i, feature in enumerate(key_features, 1):
    plt.subplot(3, 2, i)
    sns.boxplot(x=df[feature])
    plt.title(f"Box Plot {feature}")
    plt.xlabel(feature)

plt.tight_layout()
plt.show()
```



[923]: 'gas_price'

[924]:
```
# Correlation matrix
plt.figure(figsize=(10, 10))
correlation_matrix = df[key_features].corr()
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



1.0.4 Conclusion:

max_priority_fee_per_gas and ***receipt_effective_gas_price***, as well as ***gas_price***, show a high correlation with each other. This may indicate that these variables are related and likely change together.

max_fee_per_gas has a weak positive correlation with other variables, but the relationship is not strong.

gas_value has a very weak connection with other variables, indicating that changes in ***gas_value*** have little to no effect on other variables in your dataset.

2 2. Feature Engineering

2.0.1 2.1. Create new features that may be useful for predicting gas prices

- Gas prices may depend on the time of day due to changes in user activity. Gas is usually cheaper at night than during the day.
- It is used more on weekdays.
- In the winter months, it is consumed more.

[925]:
```
# Adding weights for hours, day, day of month and month weigth. + i add months␣
↪weigth because assuming higher gas prices in winter.
df["hour"] = df["date"].dt.hour
df["day_of_week"] = df["date"].dt.dayofweek
df["day_of_month"] = df["date"].dt.day
df["month"] = df["date"].dt.month
df["month_weight"] = np.where(df["month"].isin([12, 1, 2]), 1.5, 1) # Normal␣
↪month 1 - in winter 1.5
```

[926]:
```
# Deleting unnecessary columns
df = df.drop(["date", "gas_price"], axis=1)
```

3 3. Model Building and Training

3.0.1 3.1 Split the dataset into training and test sets.

[927]:
```
# Split the dataset into training and test sets
x_train, x_temp, y_train, y_temp = train_test_split(df, label, test_size=0.4,␣
↪random_state=42)
x_val, x_test, y_val, y_test = train_test_split(x_temp, y_temp, train_size=0.5,␣
↪random_state=42)
```

3.0.2 3.2 Normalize or standardize features if necessary.

[928]:
```
# Normalize columns AFTER split!
numerical_columns = ["gas_value", "max_fee_per_gas",␣
↪"max_priority_fee_per_gas", "receipt_effective_gas_price"]
scaler = StandardScaler()
x_train[numerical_columns] = scaler.fit_transform(x_train[numerical_columns])
x_val[numerical_columns] = scaler.transform(x_val[numerical_columns])
x_test[numerical_columns] = scaler.transform(x_test[numerical_columns])
```

3.0.3 3.3 Use cross-validation to tune hyperparameters for each model.

3.0.4 Train at least three different models (one of these models to be a neural network) to predict the gas price.

[929]:
```
# 1. RandomForestRegressor hyperparameter tuning
rf_model = RandomForestRegressor(random_state=42)
rf_param_dist = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5, 10]
}

random_search_rf = RandomizedSearchCV(
    estimator = rf_model,
    param_distributions = rf_param_dist,
    n_iter = 10,
    cv = 5,
    scoring = "neg_mean_squared_error",
    verbose = 1,
    n_jobs = -1,
    random_state = 42
)

random_search_rf.fit(x_train, y_train)
best_rf_model = random_search_rf.best_estimator_

# Evaluate RandomForestRegressor on validation data
rf_val_predictions = best_rf_model.predict(x_val)

rf_val_rmse = root_mean_squared_error(y_val, rf_val_predictions)
rf_val_r2 = r2_score(y_val, rf_val_predictions)
rf_val_mse = mean_squared_error(y_val, rf_val_predictions)
rf_val_mae = mean_absolute_error(y_val, rf_val_predictions)
rf_val_mape = mean_absolute_percentage_error(y_val, rf_val_predictions)
```

```
Fitting 5 folds for each of 10 candidates, totalling 50 fits
```

Why use ridge regression instead of linear regression?

Ridge regression—also known as L2 regularization—is one of several types of regularization for linear regression models.

Regularization is a statistical method to reduce errors caused by overfitting on training data

[930]:
```
# 2. Ridge Regression hyperparameter tuning
ridge_model = Ridge()
ridge_param_dist = {
    'alpha': uniform(0.1, 10)  # Regularization parameter
}
```

```
random_search_ridge = RandomizedSearchCV(
    estimator = ridge_model,
    param_distributions = ridge_param_dist,
    n_iter = 10,
    cv = 5,
    scoring = "neg_mean_squared_error",
    verbose = 1,
    n_jobs = -1,
    random_state = 42
)

random_search_ridge.fit(x_train, y_train)
best_ridge_model = random_search_ridge.best_estimator_

# Evaluate Ridge Regression on validation data
ridge_val_predictions = best_ridge_model.predict(x_val)

ridge_val_rmse = root_mean_squared_error(y_val, ridge_val_predictions)
ridge_val_r2 = r2_score(y_val, ridge_val_predictions)
ridge_val_mse = mean_squared_error(y_val, ridge_val_predictions)
ridge_val_mae = mean_absolute_error(y_val, ridge_val_predictions)
ridge_val_mape = mean_absolute_percentage_error(y_val, ridge_val_predictions)
```

```
Fitting 5 folds for each of 10 candidates, totalling 50 fits
```

There is no direct cross-validation method for neural networks using Keras/TensorFlow,

but we will use the EarlyStopping and ReduceLROnPlateau function to monitor and stop learning during retraining.

[931]:
```
# 3. Neural Network Model
neural_model = Sequential([
    Input(shape=(x_train.shape[1],)),
    Dense(64, activation="relu"),
    Dense(1)
])

learning_rate_reduction = ReduceLROnPlateau(monitor="val_loss",
                                            patience=3,
                                            verbose=1,
                                            factor=0.4,
                                            min_lr=0.00001)

earlystop = EarlyStopping(monitor="val_loss",
                          patience=5,
                          restore_best_weights=True)
```

```python
callbacks = [earlystop, learning_rate_reduction]

neural_model.compile(optimizer="adam", loss="mean_absolute_error")

neural_model.fit(x_train,
                           y_train,
                           epochs=50,
                           batch_size=64,
                           validation_data=(x_val, y_val),
                           callbacks=callbacks)

# Evaluate Neural Network on validation data
nm_val_predictions = neural_model.predict(x_val)

nm_val_rmse = root_mean_squared_error(y_val, nm_val_predictions)
nm_val_r2 = r2_score(y_val, nm_val_predictions)
nm_val_mse = mean_squared_error(y_val, nm_val_predictions)
nm_val_mae = mean_absolute_error(y_val, nm_val_predictions)
nm_val_mape = mean_absolute_percentage_error(y_val, nm_val_predictions)
```

```
Epoch 1/50
865/865              1s 902us/step -
loss: 4.3826 - val_loss: 0.2073 - learning_rate: 0.0010
Epoch 2/50
865/865              1s 844us/step -
loss: 0.1350 - val_loss: 0.0673 - learning_rate: 0.0010
Epoch 3/50
865/865              1s 852us/step -
loss: 0.0744 - val_loss: 0.0968 - learning_rate: 0.0010
Epoch 4/50
865/865              1s 812us/step -
loss: 0.0574 - val_loss: 0.0411 - learning_rate: 0.0010
Epoch 5/50
865/865              1s 835us/step -
loss: 0.0501 - val_loss: 0.0242 - learning_rate: 0.0010
Epoch 6/50
865/865              1s 820us/step -
loss: 0.0470 - val_loss: 0.0468 - learning_rate: 0.0010
Epoch 7/50
865/865              1s 825us/step -
loss: 0.0436 - val_loss: 0.0235 - learning_rate: 0.0010
Epoch 8/50
865/865              1s 817us/step -
loss: 0.0487 - val_loss: 0.0294 - learning_rate: 0.0010
Epoch 9/50
865/865              1s 788us/step -
loss: 0.0397 - val_loss: 0.0725 - learning_rate: 0.0010
Epoch 10/50
```

```
813/865            0s 658us/step -
loss: 0.0429
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0004000000189989805.
865/865            1s 855us/step -
loss: 0.0428 - val_loss: 0.1238 - learning_rate: 0.0010
Epoch 11/50
865/865            1s 825us/step -
loss: 0.0265 - val_loss: 0.0188 - learning_rate: 4.0000e-04
Epoch 12/50
865/865            1s 812us/step -
loss: 0.0184 - val_loss: 0.0202 - learning_rate: 4.0000e-04
Epoch 13/50
865/865            1s 830us/step -
loss: 0.0177 - val_loss: 0.0319 - learning_rate: 4.0000e-04
Epoch 14/50
810/865            0s 633us/step -
loss: 0.0195
Epoch 14: ReduceLROnPlateau reducing learning rate to 0.00016000000759959222.
865/865            1s 808us/step -
loss: 0.0194 - val_loss: 0.0219 - learning_rate: 4.0000e-04
Epoch 15/50
865/865            1s 794us/step -
loss: 0.0093 - val_loss: 0.0097 - learning_rate: 1.6000e-04
Epoch 16/50
865/865            1s 910us/step -
loss: 0.0091 - val_loss: 0.0060 - learning_rate: 1.6000e-04
Epoch 17/50
865/865            1s 899us/step -
loss: 0.0085 - val_loss: 0.0073 - learning_rate: 1.6000e-04
Epoch 18/50
865/865            1s 872us/step -
loss: 0.0084 - val_loss: 0.0148 - learning_rate: 1.6000e-04
Epoch 19/50
865/865            1s 831us/step -
loss: 0.0104 - val_loss: 0.0059 - learning_rate: 1.6000e-04
Epoch 20/50
865/865            1s 792us/step -
loss: 0.0076 - val_loss: 0.0167 - learning_rate: 1.6000e-04
Epoch 21/50
865/865            1s 827us/step -
loss: 0.0085 - val_loss: 0.0047 - learning_rate: 1.6000e-04
Epoch 22/50
865/865            1s 881us/step -
loss: 0.0091 - val_loss: 0.0049 - learning_rate: 1.6000e-04
Epoch 23/50
865/865            1s 790us/step -
loss: 0.0087 - val_loss: 0.0108 - learning_rate: 1.6000e-04
Epoch 24/50
```

```
863/865            0s 604us/step -
loss: 0.0089
Epoch 24: ReduceLROnPlateau reducing learning rate to 6.40000042039901e-05.
865/865            1s 802us/step -
loss: 0.0089 - val_loss: 0.0063 - learning_rate: 1.6000e-04
Epoch 25/50
865/865            1s 798us/step -
loss: 0.0043 - val_loss: 0.0047 - learning_rate: 6.4000e-05
Epoch 26/50
865/865            1s 814us/step -
loss: 0.0044 - val_loss: 0.0029 - learning_rate: 6.4000e-05
Epoch 27/50
865/865            1s 801us/step -
loss: 0.0044 - val_loss: 0.0031 - learning_rate: 6.4000e-05
Epoch 28/50
865/865            1s 849us/step -
loss: 0.0044 - val_loss: 0.0028 - learning_rate: 6.4000e-05
Epoch 29/50
865/865            1s 793us/step -
loss: 0.0047 - val_loss: 0.0054 - learning_rate: 6.4000e-05
Epoch 30/50
865/865            1s 781us/step -
loss: 0.0045 - val_loss: 0.0027 - learning_rate: 6.4000e-05
Epoch 31/50
806/865            0s 637us/step -
loss: 0.0043
Epoch 31: ReduceLROnPlateau reducing learning rate to 2.560000284574926e-05.
865/865            1s 825us/step -
loss: 0.0043 - val_loss: 0.0029 - learning_rate: 6.4000e-05
Epoch 32/50
865/865            1s 787us/step -
loss: 0.0029 - val_loss: 0.0038 - learning_rate: 2.5600e-05
Epoch 33/50
865/865            1s 797us/step -
loss: 0.0028 - val_loss: 0.0034 - learning_rate: 2.5600e-05
Epoch 34/50
865/865            1s 822us/step -
loss: 0.0029 - val_loss: 0.0025 - learning_rate: 2.5600e-05
Epoch 35/50
865/865            1s 792us/step -
loss: 0.0027 - val_loss: 0.0022 - learning_rate: 2.5600e-05
Epoch 36/50
865/865            1s 786us/step -
loss: 0.0031 - val_loss: 0.0024 - learning_rate: 2.5600e-05
Epoch 37/50
865/865            1s 826us/step -
loss: 0.0028 - val_loss: 0.0036 - learning_rate: 2.5600e-05
Epoch 38/50
```

```
865/865              1s 802us/step -
loss: 0.0033 - val_loss: 0.0020 - learning_rate: 2.5600e-05
Epoch 39/50
865/865              1s 774us/step -
loss: 0.0026 - val_loss: 0.0027 - learning_rate: 2.5600e-05
Epoch 40/50
865/865              1s 808us/step -
loss: 0.0027 - val_loss: 0.0020 - learning_rate: 2.5600e-05
Epoch 41/50
845/865              0s 681us/step -
loss: 0.0028
Epoch 41: ReduceLROnPlateau reducing learning rate to 1.0240000847261399e-05.
865/865              1s 883us/step -
loss: 0.0028 - val_loss: 0.0031 - learning_rate: 2.5600e-05
Epoch 42/50
865/865              1s 792us/step -
loss: 0.0022 - val_loss: 0.0017 - learning_rate: 1.0240e-05
Epoch 43/50
865/865              1s 798us/step -
loss: 0.0021 - val_loss: 0.0017 - learning_rate: 1.0240e-05
Epoch 44/50
865/865              1s 820us/step -
loss: 0.0023 - val_loss: 0.0017 - learning_rate: 1.0240e-05
Epoch 45/50
859/865              0s 590us/step -
loss: 0.0021
Epoch 45: ReduceLROnPlateau reducing learning rate to 1e-05.
865/865              1s 786us/step -
loss: 0.0021 - val_loss: 0.0018 - learning_rate: 1.0240e-05
Epoch 46/50
865/865              1s 814us/step -
loss: 0.0023 - val_loss: 0.0029 - learning_rate: 1.0000e-05
Epoch 47/50
865/865              1s 866us/step -
loss: 0.0023 - val_loss: 0.0017 - learning_rate: 1.0000e-05
577/577              0s 521us/step
```

[932]:
```python
# Validation results
validation_results = {
    "Model": ["Random Forest", "Ridge Regression", "Neural Network"],
    "Validation RMSE": [rf_val_rmse, ridge_val_rmse, nm_val_rmse],
    "Validation R²": [rf_val_r2, ridge_val_r2, nm_val_r2]
}
```

4 4. Model Evaluation and Selection

4.0.1 4.1 Evaluation

[933]:
```
# Predict RandomForestRegressor
rf_pred = best_rf_model.predict(x_test)

# Evaluation
rf_rmse = root_mean_squared_error(y_test, rf_pred)
rf_r2 = r2_score(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mape = mean_absolute_percentage_error(y_test, rf_pred)
```

[934]:
```
# Predict Ridge Regression
ridge_pred = best_ridge_model.predict(x_test)

# Evaluation
ridge_rmse = root_mean_squared_error(y_test, ridge_pred)
ridge_r2 = r2_score(y_test, ridge_pred)
ridge_mse = mean_squared_error(y_test, ridge_pred)
ridge_mae = mean_absolute_error(y_test, ridge_pred)
ridge_mape = mean_absolute_percentage_error(y_test, ridge_pred)
```

[935]:
```
# Predict Neural Network
nm_pred = neural_model.predict(x_test)

# Evaluation
nm_rmse = root_mean_squared_error(y_test, nm_pred)
nm_r2 = r2_score(y_test, nm_pred)
nm_mse = mean_squared_error(y_test, nm_pred)
nm_mae = mean_absolute_error(y_test, nm_pred)
nm_mape = mean_absolute_percentage_error(y_test, nm_pred)
```

```
577/577              0s 548us/step
```

The lower the MSE, RMSE, MAE, MAPE value, the better the model.

The closer the R2 value is to 1, the better the model explains the variation in the data.

[936]:
```
# Combine validation and test results
validation_results = {
    "Model": ["Random Forest", "Ridge Regression", "Neural Network"],
    "Validation RMSE": [rf_val_rmse, ridge_val_rmse, nm_val_rmse],
    "Validation R²": [rf_val_r2, ridge_val_r2, nm_val_r2],
    "Validation MSE": [rf_val_mse, ridge_val_mse, nm_val_mse],
    "Validation MAE": [rf_val_mae, ridge_val_mae, nm_val_mae],
    "Validation MAPE": [rf_val_mape, ridge_val_mape, nm_val_mape]
}
```

```
test_results = {
    "Model": ["Random Forest", "Ridge Regression", "Neural Network"],
    "Test RMSE": [rf_rmse, ridge_rmse, nm_rmse],
    "Test R²": [rf_r2, ridge_r2, nm_r2],
    "Test MSE": [rf_mse, ridge_mse, nm_mse],
    "Test MAE": [rf_mae, ridge_mae, nm_mae],
    "Test MAPE": [rf_mape, ridge_mape, nm_mape]
}

validation_df = pd.DataFrame(validation_results)
test_df = pd.DataFrame(test_results)

# Merge validation and test results
result = pd.merge(validation_df, test_df, on="Model")
```

4.0.2 4.2 Compare the models and select the best one based on performance metrics

[937]:
```
# Compare the models
result = pd.DataFrame(result)
result.round(6)
```

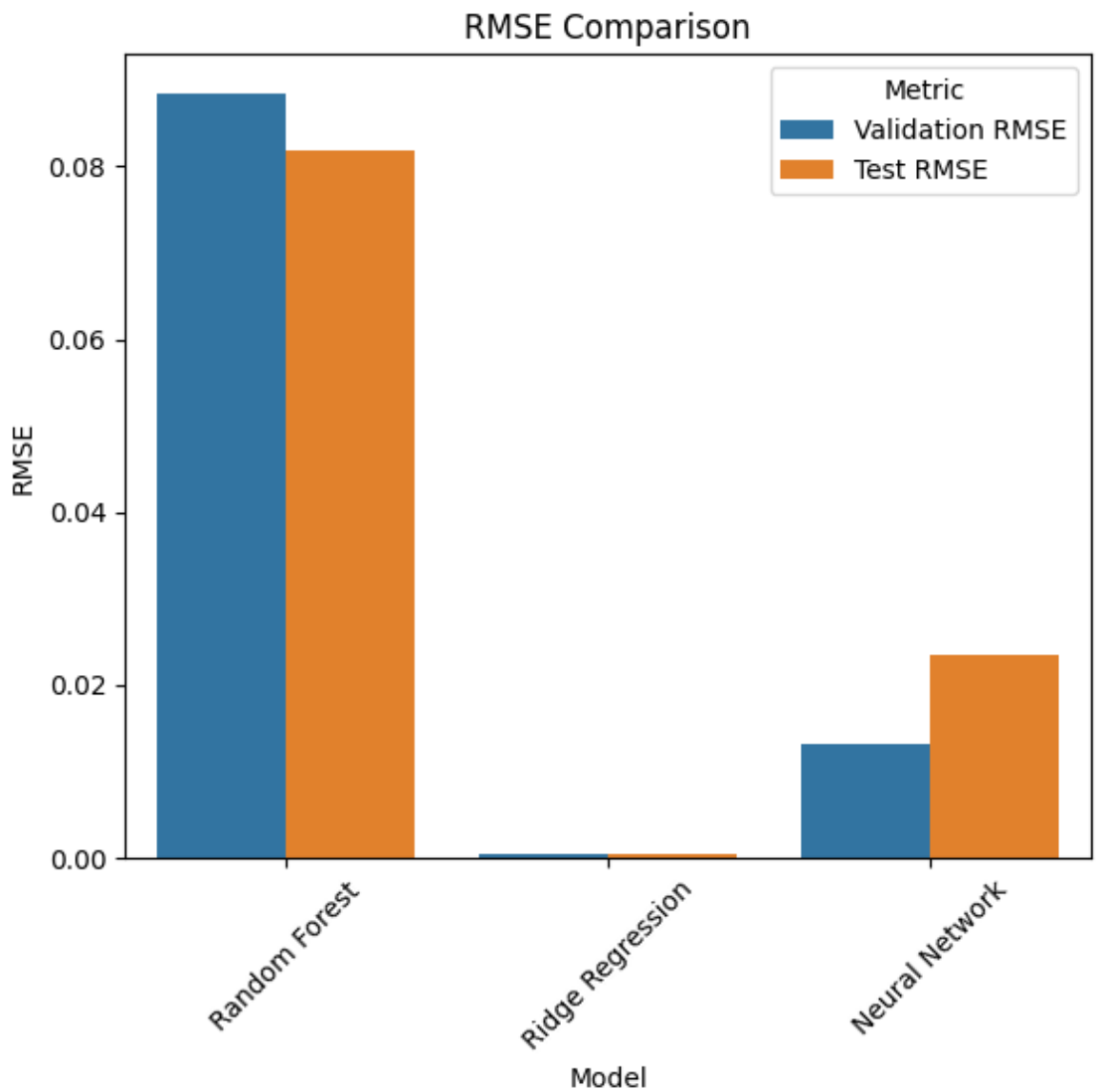
[937]:
```
              Model  Validation RMSE  Validation R²  Validation MSE  \
0     Random Forest         0.088527       0.999920        0.007837
1  Ridge Regression         0.000408       1.000000        0.000000
2    Neural Network         0.013275       0.999998        0.000176

   Validation MAE  Validation MAPE  Test RMSE   Test R²  Test MSE  Test MAE  \
0        0.003137         0.000069   0.081850  0.999926  0.006699  0.003145
1        0.000213         0.000030   0.000422  1.000000  0.000000  0.000214
2        0.001665         0.000217   0.023404  0.999994  0.000548  0.001865

   Test MAPE
0   0.000068
1   0.000030
2   0.000212
```

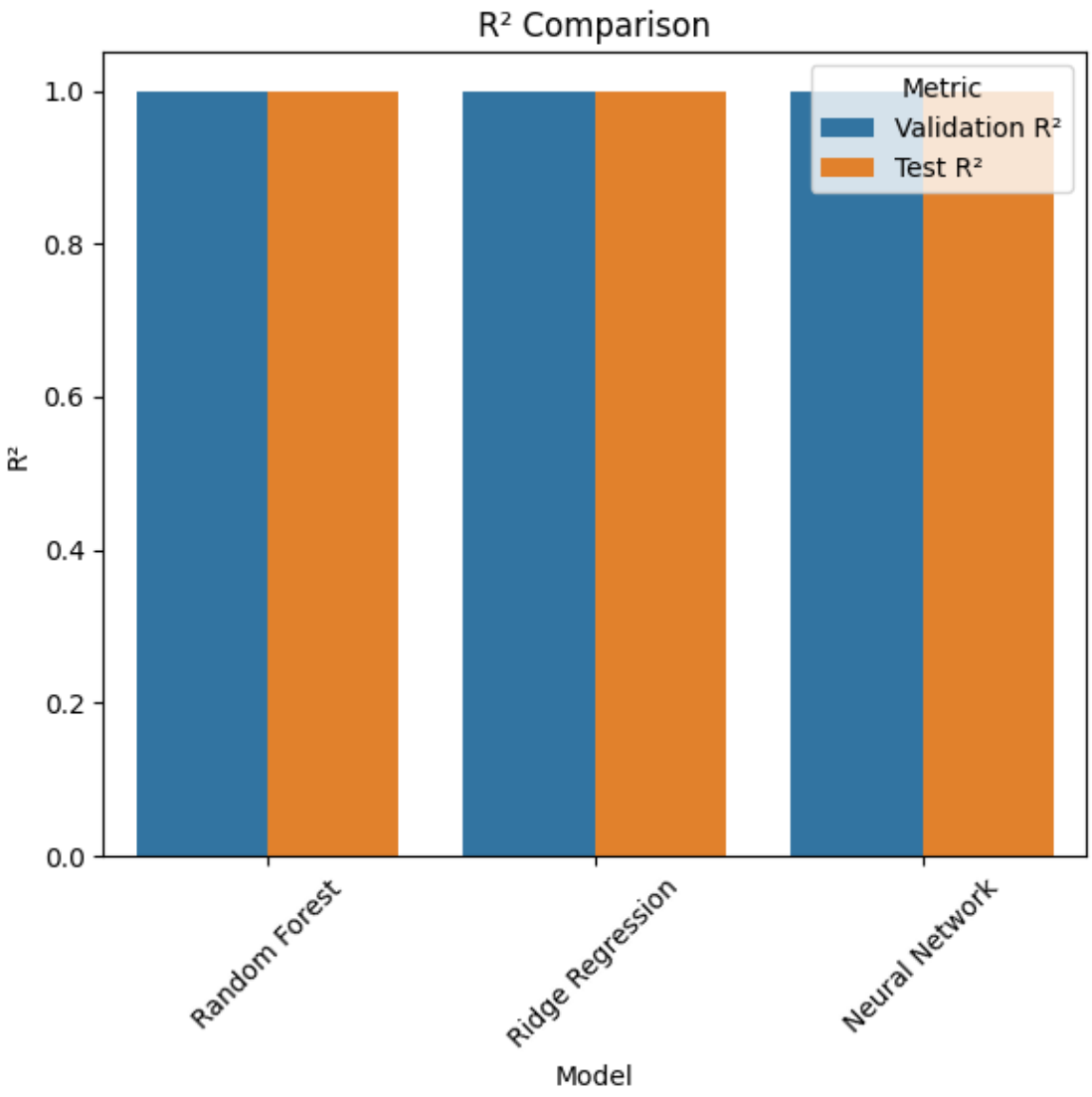
The Random Forest result R^2 value (0.999926) and relatively low RMSE (0.081850) indicates that the model effectively explains the variation in the data, although prediction errors still occur.

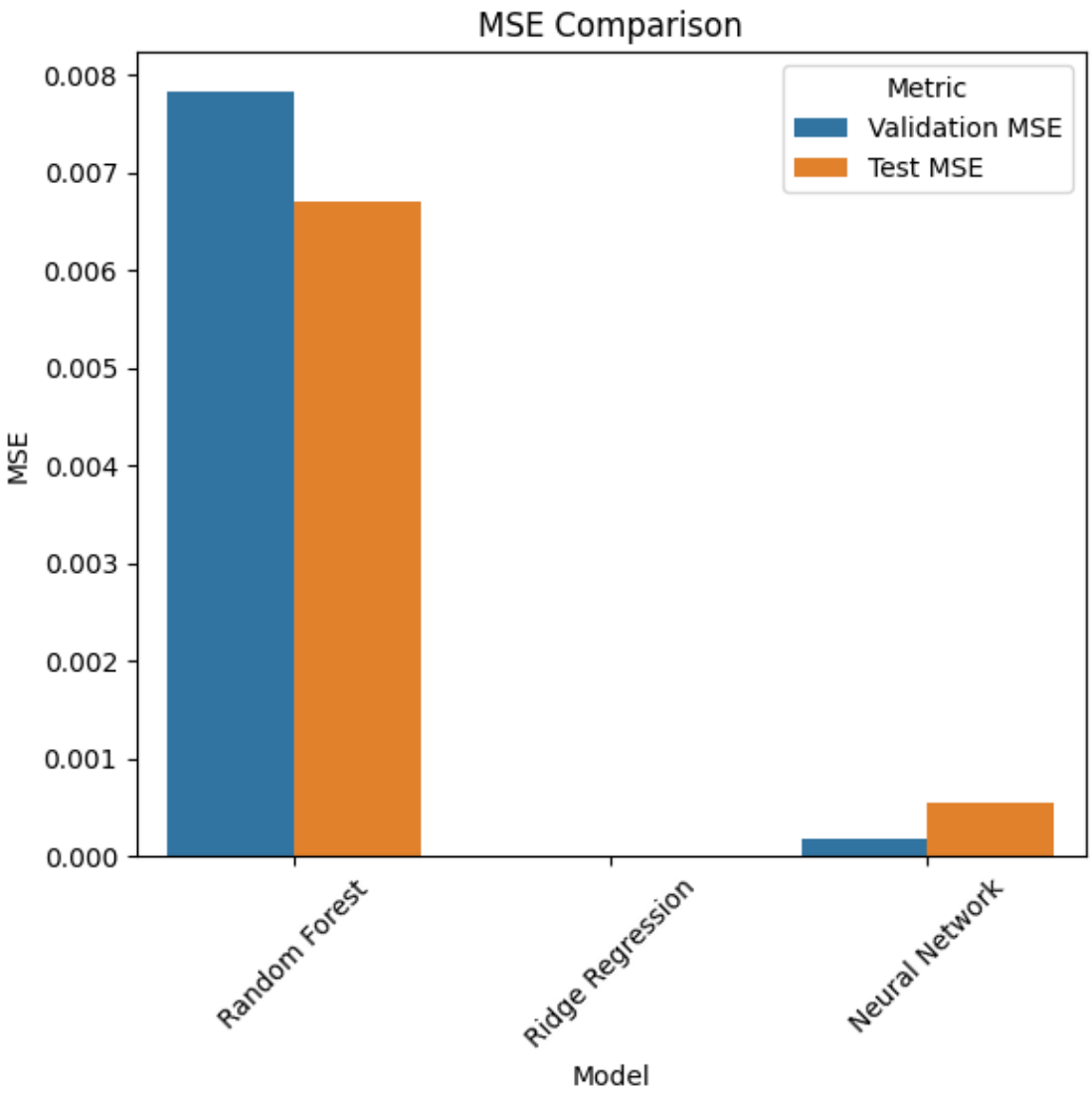
The Ridge Regression has demonstrated the best results among all models R^2 value (1.000000) and the lowest RMSE (0.000422). These results indicate that linear regression with regularization copes with the task very effectively.

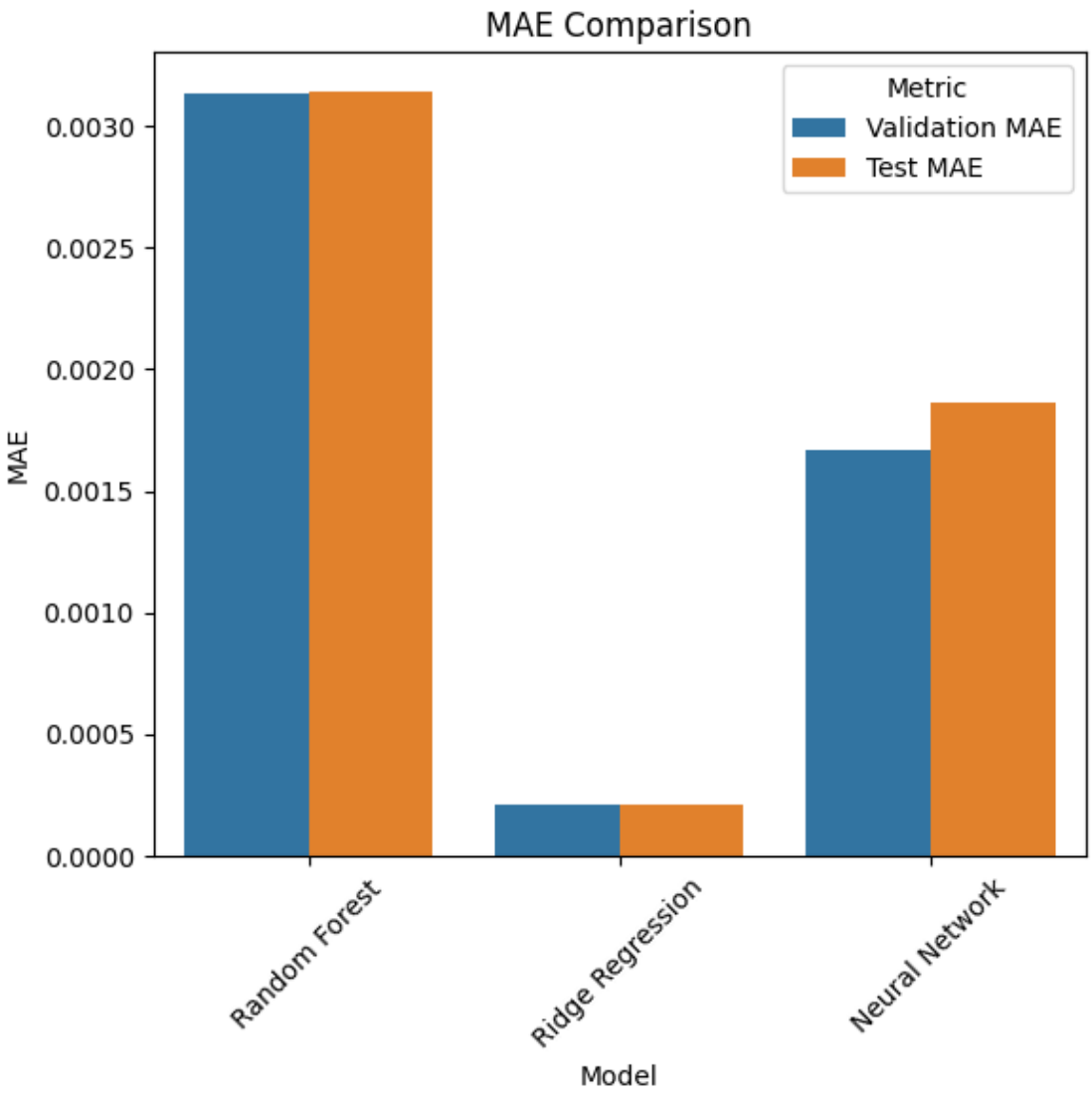
Neural Network - RMSE (0.028137) and MSE (0.000792) has shown satisfactory results, but they are slightly lower compared to Ridge Regression. This indicates that the model needs additional fine-tuning, but the model itself can work much better on big data.

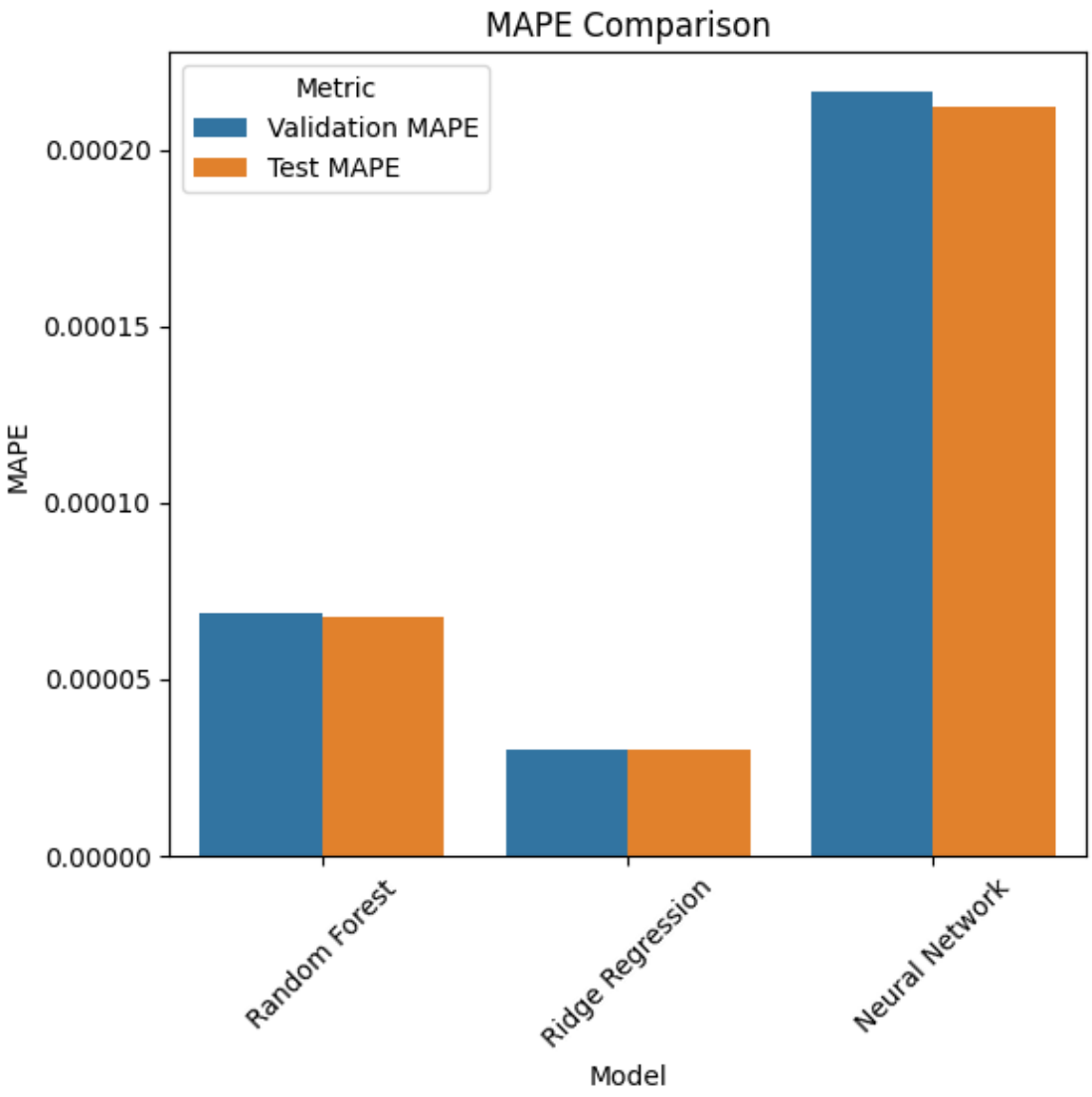
[940]:
```
for metric in ["RMSE", "R²", "MSE", "MAE", "MAPE"]:
    plt.figure(figsize=(6, 6))
    sns.barplot(data=result.melt(id_vars="Model", value_vars=[f"Validation␣
↪{metric}", f"Test {metric}"], var_name="Metric", value_name="Score"),
                x="Model", y="Score", hue="Metric")
    plt.title(f"{metric} Comparison")
    plt.ylabel(f"{metric}")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```




R² Comparison
Metric
Validation R²
Test R²
1.0
0.8
0.6
0.4
0.2
0.0
R²
Random Forest
Ridge Regression
Neural Network
Model


MSE Comparison
Metric
Validation MSE
Test MSE
MSE
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
Random Forest
Ridge Regression
Neural Network
Model


MAE Comparison
Metric
Validation MAE
Test MAE
0.0030
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
MAE
Random Forest
Ridge Regression
Neural Network
Model


MAPE Comparison
Metric
Validation MAPE
Test MAPE
0.00020
0.00015
0.00010
0.00005
0.00000
MAPE
Random Forest
Ridge Regression
Neural Network
Model